Cloud Courier: final submission

CSC 4360 / Mobile App Development / Parsh Jadon

Repository: https://github.com/parshjadon/MADinc14

Device: Samsung SM-S938U1 / Android 17 / Permission: authorized. Package: com.parshjadon.cloud_courier / Firebase project: temp-fc580.

Three required reflection responses

1. Setup and payload

The app targets Android with a separately registered package and FlutterFire-generated options to prevent project mismatches. Firebase initialization finishes before messaging registration and widget startup. Core 4.x and Messaging 16.x retain the course's token API. The string `action=update_ui` permits an update; `asset=default` or `asset=promo` selects a bundled image. The notification title supplies the heading. An allowlist avoids arbitrary asset paths. A token addresses an installation; delivery does not authorize access to protected resources.

2. Lifecycle and evidence

`onMessage` handles foreground receipt, `onMessageOpenedApp` handles a background tap, and `getInitialMessage()` handles a notification-triggered cold start. Android executes the separate `onBackgroundMessage` callback in another isolate, preventing direct widget updates. T1 shows `onMessage`, count 1, and promo content; T2 shows the notification followed by `onMessageOpenedApp`, count 2; T3 shows `getInitialMessage`, count 1, and expected content on a new launch. These notification-plus-data results do not establish data-only reliability. Android force-stop blocks delivery until reopening; one phone cannot establish universal platform behavior.

3. Validation and next step

T4 changes `asset` to `unknown`: its screenshot shows rejection, `onMessage`, and count 2 while preserving T3's title/image. Lifetime guards prevent callbacks from changing disposed widgets; disposal cancels listeners. Repeated content is harmless here, but business operations would benefit from event-ID deduplication: sending the same ID twice should produce one operation. AI assisted the code, automated tests, and writing; the student supplied T1-T4 device screenshots.

All four device tests passed. Screenshots were supplied by the student. AI assisted the implementation, automated tests, and writing. The full device token is not shown or included. LMS submission remains a separate action.

Payload and four-test record

CSC 4360 / Mobile App Development / Parsh Jadon

Valid message content for T1-T3. A trusted sender supplies the target token and HTTP v1 envelope when required.

```
{
  "notification": {
    "title": "Campus event update",
    "body": "See the latest campus event announcement"
  },
  "data": {
    "action": "update_ui",
    "asset": "promo"
  }
}
```

T4 variation: change only data.asset from promo to unknown; keep action=update_ui.

Test	Expected path/result	Observed device result	Status	Evidence
T1	onMessage; update title/image	Count 0 -> 1; Campus event update and promo image; Applied payload: asset=promo.	PASS	Page 3
T2	onMessageOpenedApp after notification tap	Notification delivered; open handler shown; count 2; expected event title and promo image.	PASS	Page 4
T3	getInitialMessage after cold-start tap	Notification delivered; initial-message handler shown; count 1 on new launch; expected content.	PASS	Page 5
T4	onMessage; reject unknown asset safely	Count 1 -> 2; payload rejected; previous title/image preserved; app remains on screen.	PASS	Page 6

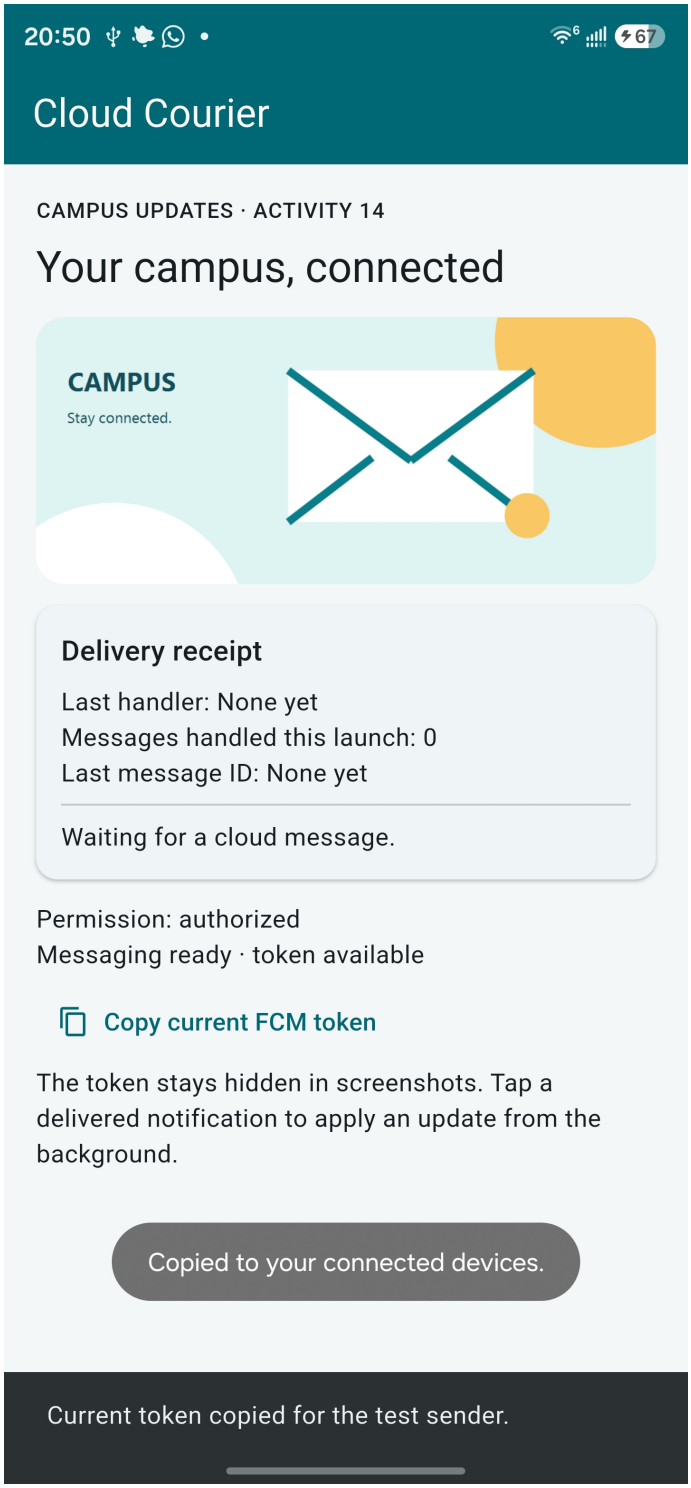
Checks: flutter analyze - no issues; flutter test - 3 passed; flutter build apk --debug - successful. The original screenshots demonstrate device behavior; automated fake-service tests alone do not prove FCM delivery.

Limits: one Samsung/Android target, notification-plus-data payloads. No second-platform, data-only background-delivery, or universal reliability claim is made. Background processing was registered separately from notification taps.

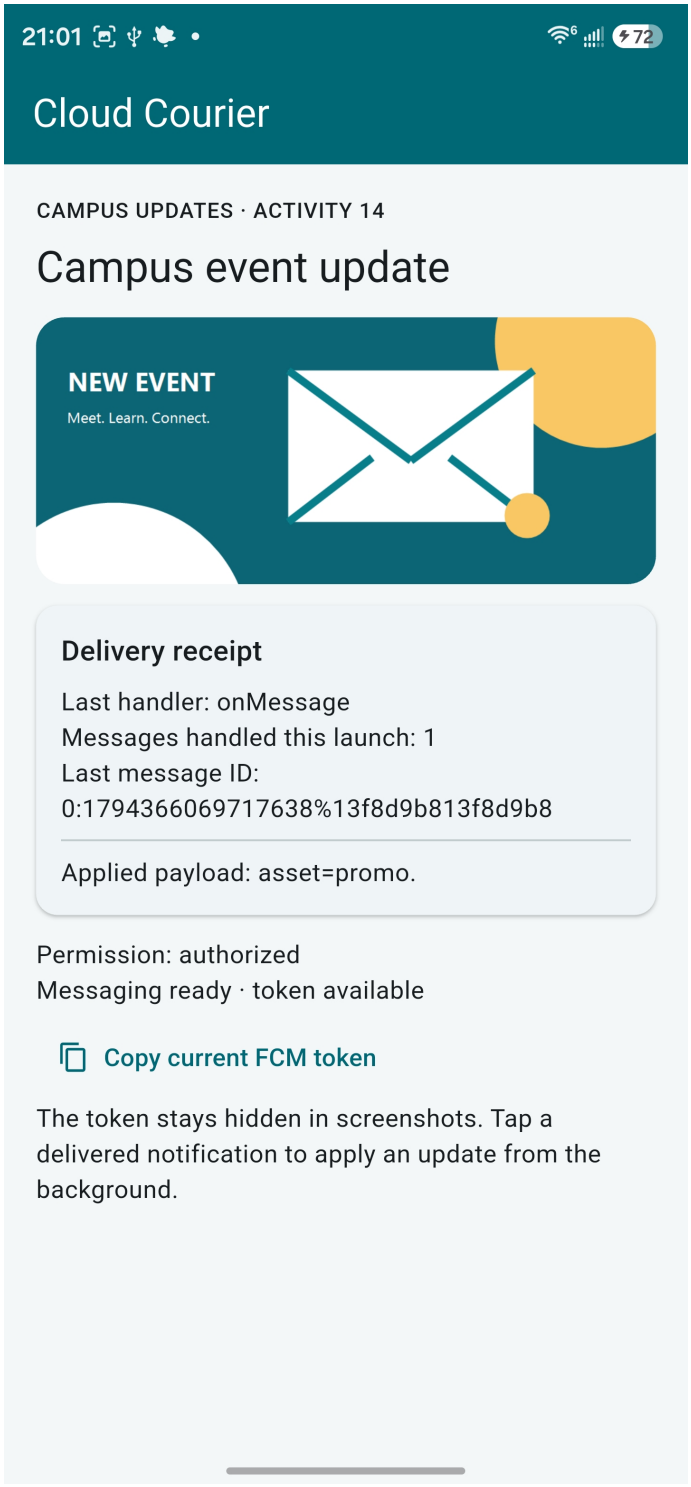
T1 - Foreground receipt

Composite evidence / Original screenshots arranged without changing app content

Before message: default content / count 0



After message: onMessage / count 1

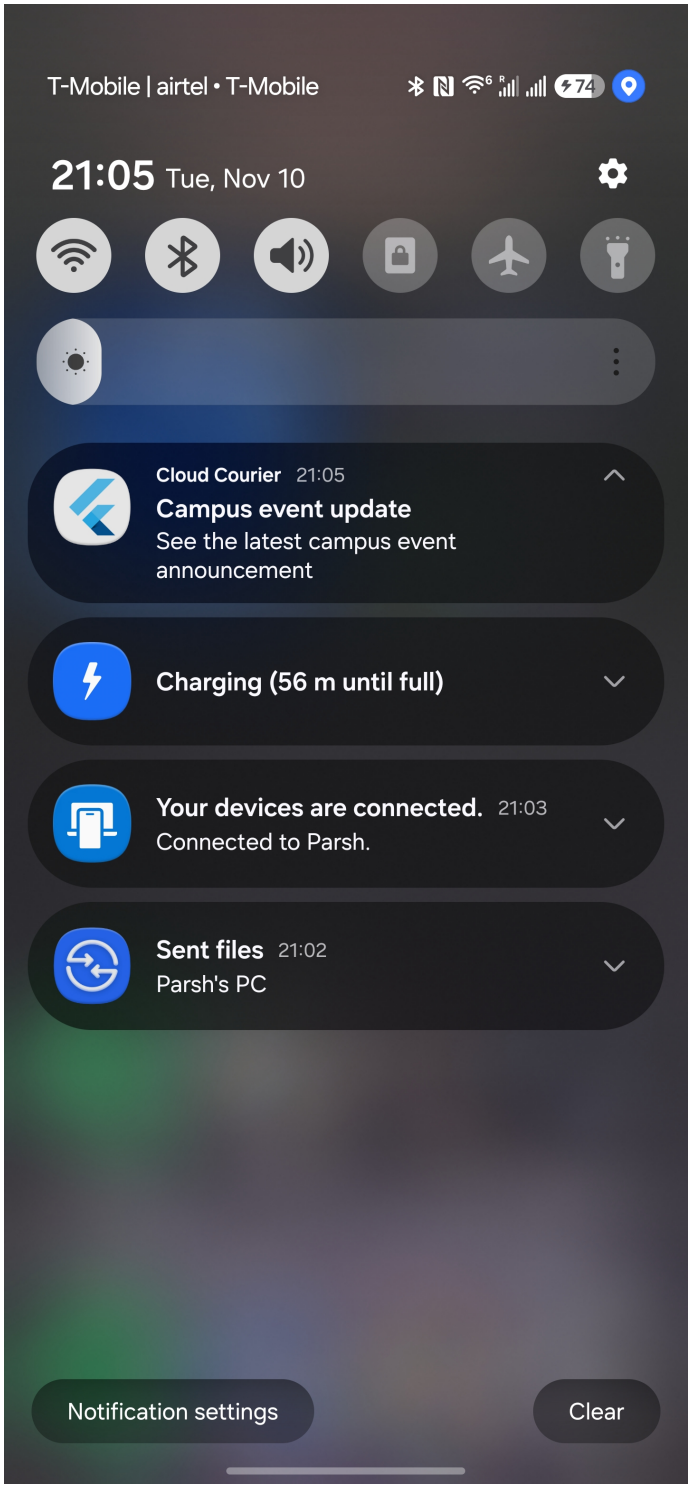


PASS: the title changes to Campus event update; the promo image replaces default content.

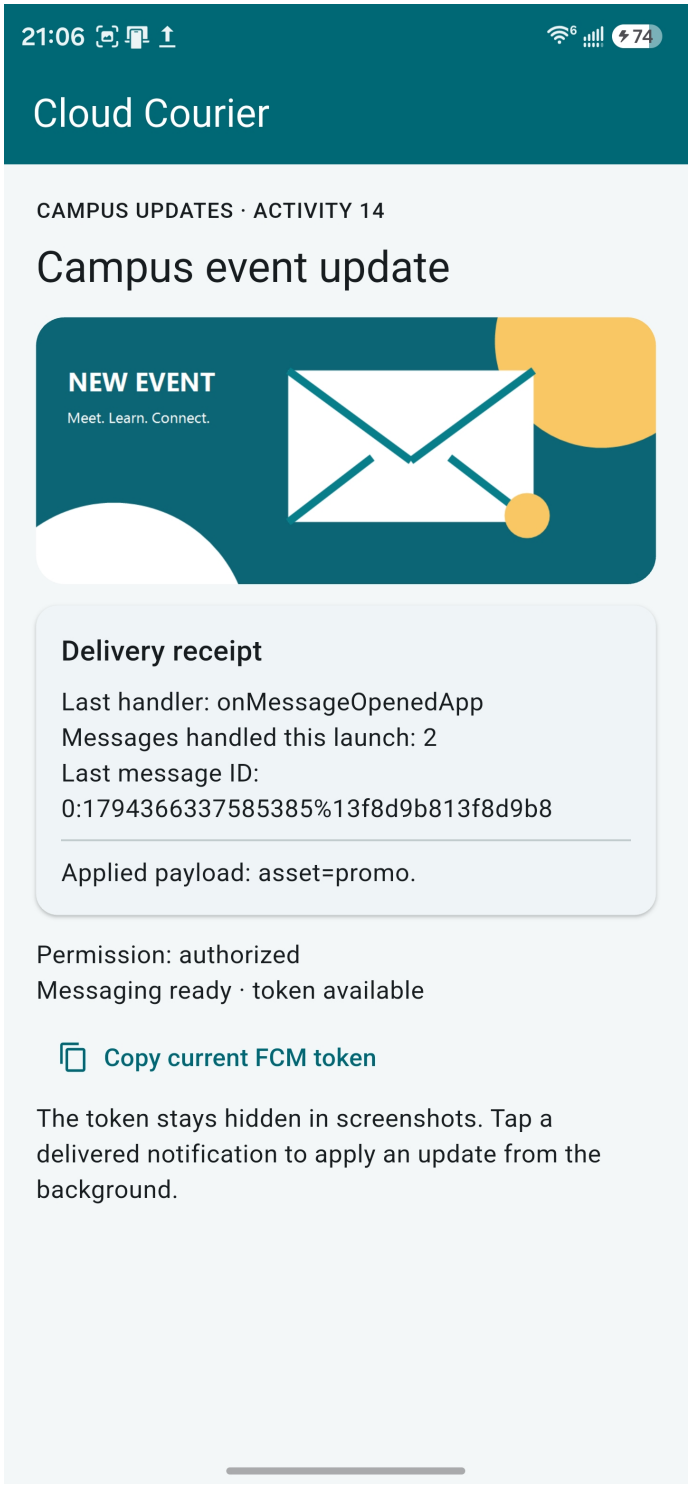
T2 - Background notification tap

Composite evidence / Original screenshots arranged without changing app content

Delivered notification before tap



After tap: onMessageOpenedApp / count 2

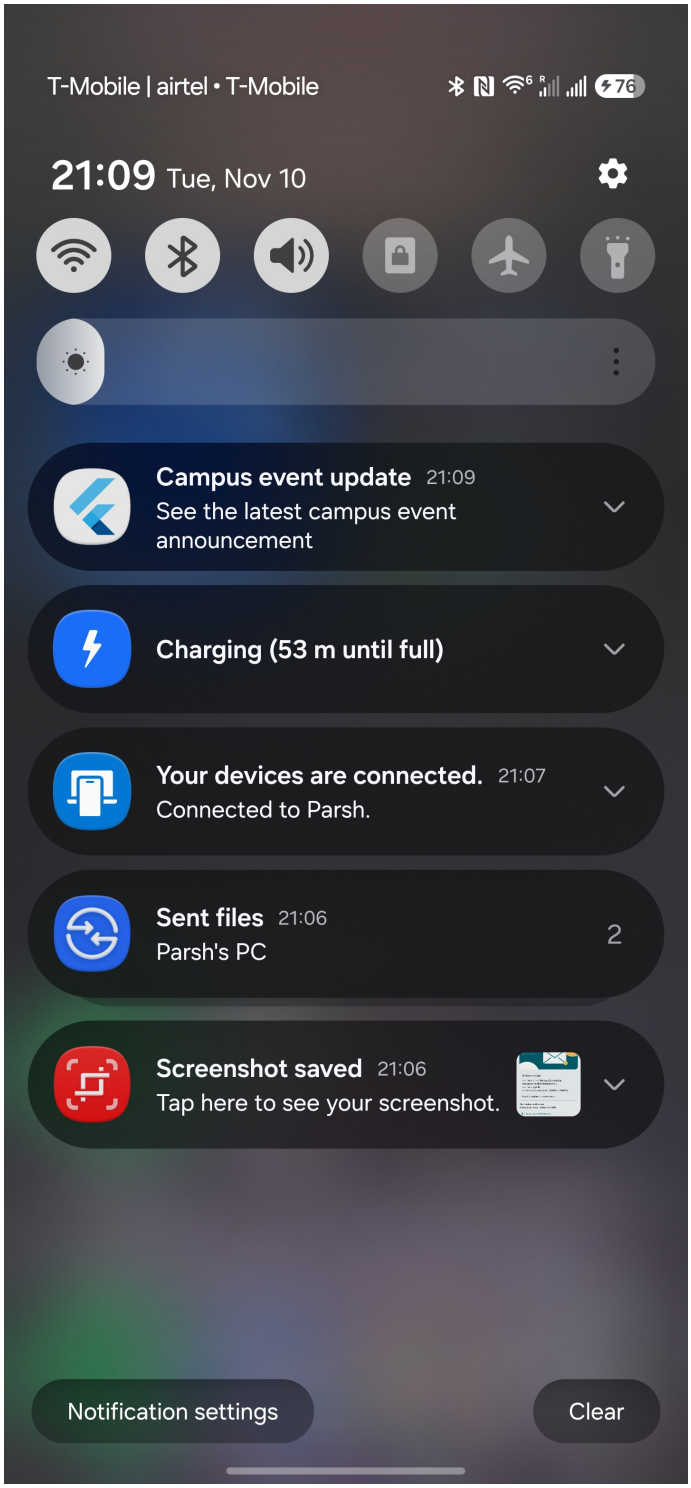


PASS: the delivered notification opens the app; the fresh handler/count establishes a new interaction.

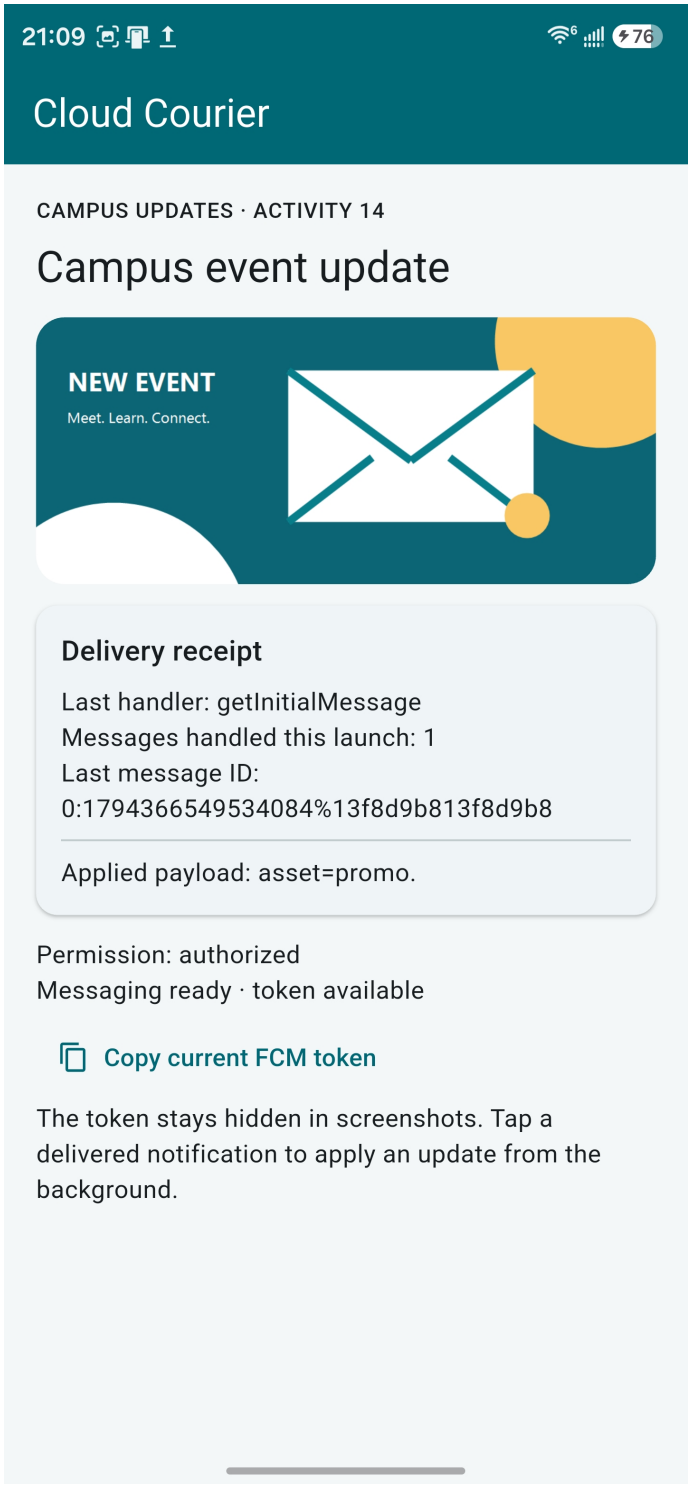
T3 - Notification-triggered cold start

Composite evidence / Original screenshots arranged without changing app content

Delivered notification before launch



After tap: getInitialMessage / count 1

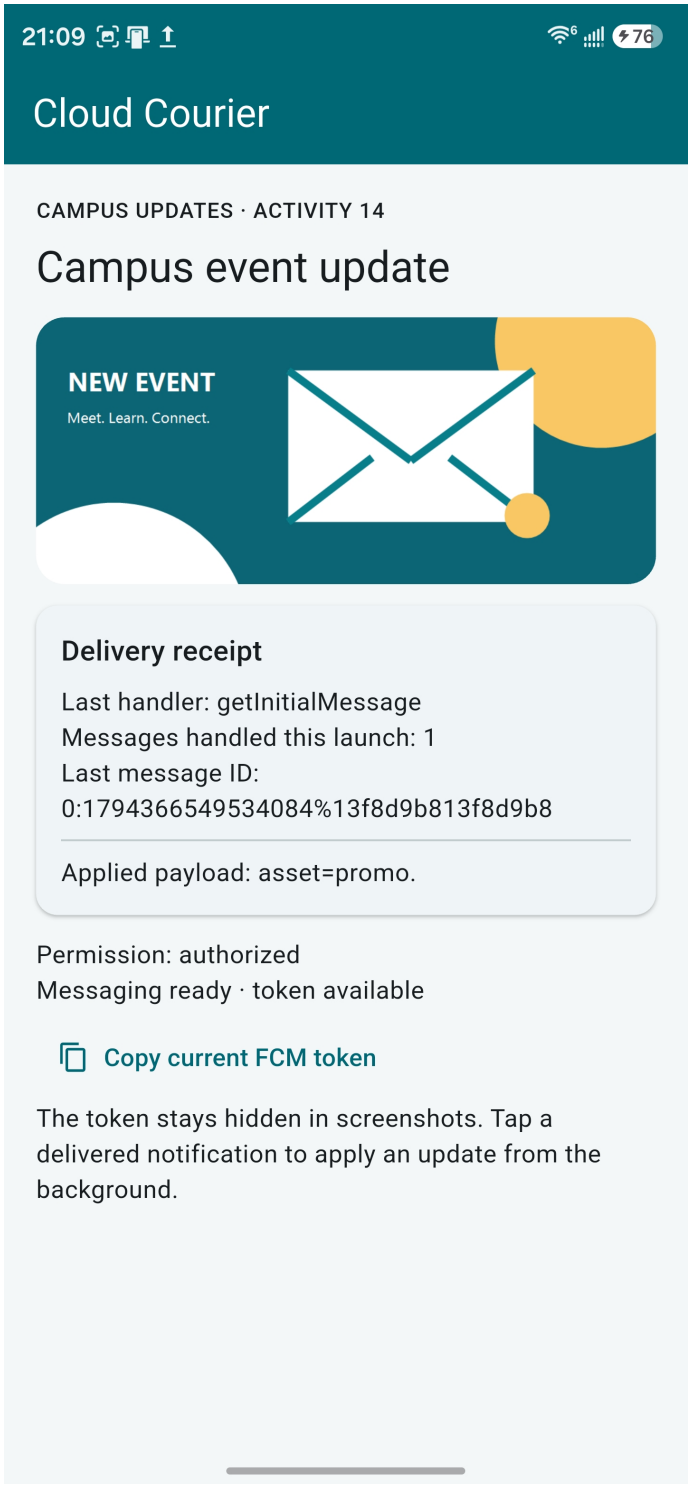


PASS: a new launch consumes the initial message and applies the expected event content.

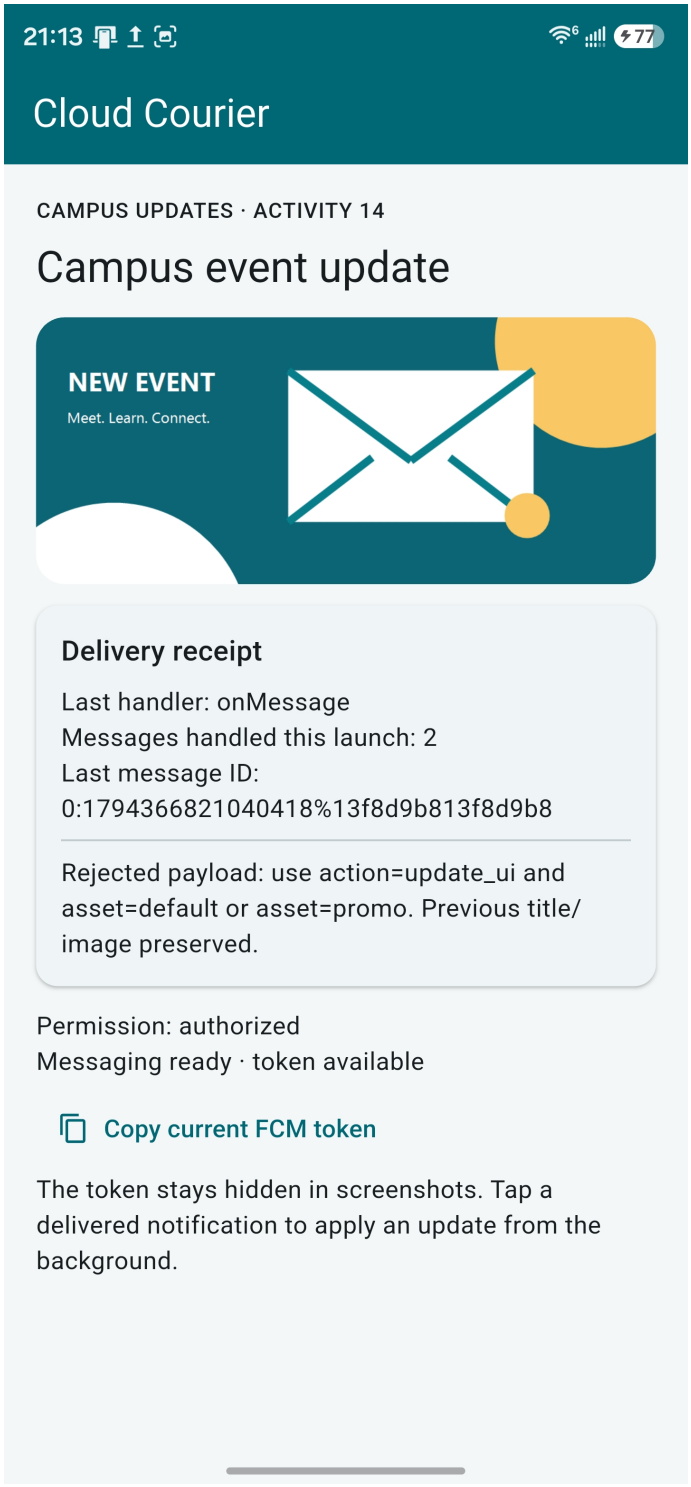
T4 - Invalid payload rejected

Composite evidence / Original screenshots arranged without changing app content

Before: reuse T3 content / count 1



After: onMessage / rejection / count 2



PASS: asset=unknown is rejected. Previous event title/promo image survive while receipt metadata advances.

Practice questions and answers

CSC 4360 / Mobile App Development / Parsh Jadon

Practice Arena answers

1. Foreground receipt uses `FirebaseMessaging.onMessage`.

2. Check `getInitialMessage()` after Firebase initialization for a notification-triggered launch from termination.

3. Reject unknown assets against the allowlist, explain the diagnostic, and preserve safe UI.

4. Strong background evidence connects the payload, actual notification tap, matching handler/count, and expected visible result.

Implementation knowledge-check answers

1. The background callback is top-level so the plugin can invoke it without a widget instance; Android background execution uses a separate isolate. The entry-point annotation keeps it available in optimized builds.

2. `getInitialMessage()` checks whether a notification tap launched a terminated app.

3. Change a simple label or image from the validated payload before adding complexity.

4. A reinstall may change the token; retrieve the current token before the next send.

Optional reasoning and disclosure

Token Tracker: Tokens identify current app installations and can rotate. Using an old token can target a removed installation. The service observes token refresh.

State Mapper: Visible app -> receipt -> `onMessage` -> validated widget update. Background app -> delivered notification -> tap -> `onMessageOpenedApp` -> widget update. Terminated app -> delivered notification -> tap/launch -> `getInitialMessage()` -> widget update. Separately delivered background work -> top-level `onBackgroundMessage` -> bounded non-UI processing; it is not a notification-open event.

Payload Validator: T4's device result and its comparison with T3 demonstrate rejection and preservation. Automated tests also cover missing/malformed assets.

Evidence Builder: The JSON and four rows above define reproducible device checks. A send confirmation or build success alone would not prove receipt or a UI reaction.

Design Critic: The claim that every background notification immediately changes widgets is incorrect. Firebase distinguishes background work from notification-open handling. T2's notification and `onMessageOpenedApp` screenshots support the background-tap path. T3 separately shows the cold-start `getInitialMessage` result. See Firebase's message-handling guide (https://firebase.google.com/docs/cloud-messaging/flutter/receive-messages).

Repeat-send trade-off: Reapplying the same content is harmless in this sample; receipt metadata still advances. A production workflow with side effects should use an application event ID and durable deduplication, tested by sending the same event twice and observing exactly one business operation.

Optional payload detail: Both required custom values are strings. Unsupported actions, missing assets, unknown asset names, and non-string asset values are rejected. The notification body is informational; the two data keys determine the UI action.

Optional improvement detail: With all four device results recorded, add event-ID deduplication if repeated notifications trigger business operations. Measure success through one operation for repeated IDs and separate operations for distinct IDs.

Assistance disclosure: Codex generated the code, original geometric image assets, automated tests, and these written answers from the supplied assignment. Review and disclose that assistance according to the course's policy.